1 Introduction

1.1 Overview and Motivation

Software-intensive systems increasingly operate in highly regulated environments where ensuring compliance with laws, standards, and regulatory directives has become an essential part of software development [1, 2]. Organizations in domains such as finance, insurance, healthcare, and public administration cannot treat compliance as an afterthought, but must continuously adapt their software systems to evolving regulatory requirements while maintaining software quality, development efficiency, and business continuity. As regulatory frameworks continue to increase in complexity and evolve over time, ensuring regulatory compliance has become an increasingly important challenge for software engineering research and practice, motivating extensive research on Regulatory Requirements Engineering, compliance management, compliance automation, and AI-assisted support for regulatory activities [3, 4, 5].

Maintaining regulatory compliance is not a one-time activity but a continuous engineering process driven by the ongoing evolution of regulations. Consequently, organizations must continuously monitor regulatory updates, identify relevant changes, understand their implications, and determine how these changes affect existing software systems and development activities. These tasks are often performed manually through document comparison, expert analysis, and iterative discussions, making the process time-consuming, error-prone, and difficult to scale in rapidly evolving regulatory environments [6, 7]. Once a regulatory change has been identified, its relevance and engineering implications must be assessed, since not every change necessarily requires a corresponding modification of the software system. Changes to regulatory sources may introduce or modify obligations, clarify existing provisions, or alter previously applicable rules. Their interpretation may consequently require changes to engineering specifications and related software artifacts, or reveal inconsistencies and conflicts that need to be resolved. Similar to the principles of Continuous Software Engineering, organizations must therefore continuously reassess and evolve their software artifacts throughout the software lifecycle in response to relevant regulatory changes [8, 9].

Recent advances in Natural Language Processing (NLP) and, more recently, Large Language Models (LLMs) have demonstrated considerable potential for supporting software engineering tasks involving natural language artifacts, including requirements engineering, software maintenance, testing, and documentation [10, 11]. Within the regulatory compliance domain, AI-assisted approaches have been investigated for extracting regulatory requirements, rights, obligations, and other legal information [12, 13], compliance checking [14], and question answering over compliance requirements [15]. More recently, AI-assisted techniques have been explored for regulatory change analysis [16] and LLM-based requirements change impact analysis [17]. Recent secondary studies further show that research on regulatory compliance and Regulatory Requirements Engineering spans a broad range of methods and activities, but remains fragmented across individual tasks and application contexts [3, 5]. Nevertheless, the current body of knowledge provides limited integrated support, with most approaches addressing isolated tasks rather than the complete engineering process surrounding evolving regulations. In particular, relatively little attention has been devoted to systematically supporting engineers in understanding regulatory changes, assessing their significance, and analyzing their impact on software artifacts [3, 5]. Motivated by these challenges, this dissertation analyzes the challenges engineers face in regulatory change analysis and impact assessment and investigates the extent to which these challenges can be addressed through automated and LLM-assisted approaches.

Rather than viewing artificial intelligence as a replacement for engineering expertise, this research follows a human-in-the-loop perspective in which LLMs and automated methods support engineers by automating repetitive analysis tasks, improving consistency, and supporting informed decision-making while keeping practitioners responsible for the final interpretation and implementation decisions. To

that end, the dissertation adopts Design Science Research as an overarching methodological approach, combining systematic literature studies, empirical investigations in industrial settings, solution design and implementation, and tool-supported evaluations. The resulting body of work contributes empirically backed LLM-assisted methods and tool-supported approaches for supporting regulatory change analysis and impact assessment in software systems.

The research presented in this dissertation was conducted in close collaboration with an industrial partner in the financial sector. This collaboration constitutes an important part of the relevance cycle of the adopted Design Science Research approach, grounding the identification of research problems, the development of solutions, and their evaluation in industrial practice. Through this interplay between industrial needs and research activities, the dissertation contributes empirically backed LLM-assisted RE methods for regulatory change analysis and impact assessment.

The remainder of this kappa is organized as follows. Section 1.2 introduces the theoretical background and related work on regulatory compliance, regulatory change analysis, compliance automation, and AI-assisted software engineering. Sections 1.3 and 1.4 identify the research gaps that motivate this dissertation and present the research objectives and research questions. Section 1.5 describes the overall research methodology and explains how the individual studies are connected within the research program. Section 1.6 synthesizes the contributions of the included papers and explains how they collectively advance AI-assisted regulatory change analysis and impact assessment. Section 1.7 discusses the overall findings, implications for research and practice, limitations, and directions for future work before we conclude in Section 1.8.

1.2 Background

This section introduces the theoretical foundations that underpin this dissertation. It first presents Regulatory Requirements Engineering as the discipline concerned with supporting software compliance by design. Then, it discusses the challenges associated with evolving regulations and regulatory change analysis before reviewing recent advances in AI-assisted techniques to support regulatory engineering activities.

1.2.1 Regulatory Requirements Engineering

Software-intensive systems increasingly operate in environments governed by laws, standards, and regulatory directives. As the number, complexity, and scope of regulations continue to grow, organizations are required to ensure that their software systems comply with evolving regulatory obligations. Regulatory compliance has therefore become an important research area within software engineering and information systems, extending beyond legal concerns to become an engineering challenge that affects the software development lifecycle (SDLC) [18, 1, 2, 19].

Regulatory Requirements Engineering (RRE) has emerged as a specialized area within Requirements Engineering that investigates how regulations can be systematically considered during software development [20]. While regulators may be considered stakeholders and regulatory obligations have long influenced software development, regulatory sources have often received less explicit attention in requirements engineering compared with customer, user, and business needs. RRE places particular emphasis on systematically identifying, interpreting, and integrating regulatory requirements into software engineering processes. Existing work has proposed methods for acquiring regulatory requirements, modeling legal constraints, checking compliance, and linking regulatory obligations to software artifacts [1, 2, 21, 6].

Despite these advances, regulatory compliance remains a challenging engineering problem. Regulations are typically expressed in natural language, often containing complex structures, domain-specific terminology, and implicit dependencies across multiple documents. Furthermore, regulatory engineering is inherently multidisciplinary, involving legal, business, and technical perspectives. As a result, organizations continue to rely heavily on manual analyses and expert judgment when implementing and maintaining regulatory compliance. More recently, the principle of *compliance by design* has emphasized that compliance should not be treated as a late verification activity, but rather be integrated upfront and throughout the software development lifecycle, from requirements engineering to software architecture, implementation, testing, and maintenance [20].

Recent secondary studies indicate that research in Regulatory Requirements Engineering remains fragmented, with individual studies often focusing on isolated engineering activities, specific regula-

tions, or particular application domains. Moreover, relatively few approaches provide comprehensive support throughout the entire regulatory engineering process or evaluate their solutions in industrial settings [3, 5].

1.2.2 Regulatory Change Analysis

Regulatory Requirements Engineering is inherently evolutionary. Regulatory requirements continuously evolve as regulations are amended, extended, clarified, or replaced. For example, official EU legislative statistics report thousands of legislative acts and amendments adopted annually, illustrating the dynamic nature of modern regulatory environments [22]. Similar trends have been observed in other jurisdictions, where regulatory databases show a sustained increase in the amount and complexity of regulation over time [23]. Consequently, maintaining compliance requires organizations not only to understand applicable regulations but also to continuously monitor regulatory updates and assess their consequences for existing software systems.

Regulatory change analysis encompasses the activities required to identify modifications between successive versions of regulatory documents, interpret their meaning, determine their relevance, and assess their impact on software artifacts and engineering activities. These activities are central to regulatory compliance because changes in regulations may require corresponding changes to requirements, workflows, architecture, implementation, documentation, test cases, or quality assurance activities [24, 25, 26].

It is important to distinguish regulatory change and impact analysis from runtime compliance monitoring or enforcement. Runtime compliance approaches assess whether an executing software system or process conforms to applicable regulatory constraints during operation. In contrast, the focus of this dissertation is on the evolution and maintenance of existing software systems in response to regulatory change. When regulations evolve, engineers need to determine what has changed, interpret its engineering relevance, and identify which existing software artifacts may require adaptation. Regulatory impact analysis in this dissertation therefore concerns a broader set of engineering artifacts, including requirements, workflows, architecture, implementation, documentation, and tests, rather than compliance checking of an executing system at runtime.

Software change impact analysis frequently relies on structured or explicitly linked software artifacts, such as requirements models, traceability links, source code dependencies, or architectural models [24, 25, 27]. Regulatory change analysis, however, must often operate on natural language documents that contain implicit references, heterogeneous structures, domain-specific terminology, and limited information about the rationale or implementation consequences of individual changes [1, 5, 26].

Industrial evidence further suggests that regulatory change handling is often manual and time-consuming. Practitioners have to compare document versions, inspect change logs, interpret regulatory updates, identify affected requirements or system components, and coordinate with domain experts to determine implementation actions [7, 26]. Related work has also reported challenges in maintaining documentation, tracking requirements changes, supporting traceability, and ensuring testing and verification under evolving regulations [27].

Recent work has started to investigate regulatory change analysis more directly. Jayatilleke et al. propose a framework for specifying and classifying requirements changes to support communication between stakeholders [24]. Abualhaija et al. propose MURCIA, an AI-enabled approach for analyzing regulatory changes across textual, semantic, and deontic layers [16]. Barrientos et al. investigate impact analysis of regulatory requirement changes on business process compliance [28]. More recently, Etezadi et al. propose ProReFiCIA, an LLM-driven approach for inter-requirement change impact analysis that identifies requirements affected by natural-language change rationales and combines prompt engineering, refinement, filtering, and retrieval-augmented generation [17]. These studies indicate increasing interest in automated change impact analysis. However, they address different points in the change-management process. In particular, inter-requirement impact analysis assumes that a software requirements change or change rationale is already available, whereas regulatory change analysis must additionally identify and interpret changes originating in external regulatory documents and determine how they propagate to heterogeneous software engineering artifacts. A further challenge is that regulatory changes differ in their relevance, context, and potential criticality. Treating all detected changes uniformly may therefore generate impact candidates without sufficiently distinguishing which changes are relevant to a particular software system or require closer engineering attention. Effective regulatory impact analysis consequently requires not only identifying potential dependencies, but

also contextualizing and prioritizing changes according to their engineering significance. Systematic support for this broader regulatory change-to-software impact process therefore remains limited.

1.2.3 AI-supported Regulatory Engineering

Recent advances in Artificial Intelligence (AI), particularly Natural Language Processing (NLP) and Large Language Models (LLMs), have expanded the possibilities for supporting regulatory engineering activities. NLP techniques have been applied to legal information extraction, document classification, semantic similarity analysis, traceability recovery, compliance checking, and requirements extraction [12, 13, 25, 14, 15]. Domain-specific language models, such as Legal-BERT, and recent studies on LLMs for legal text processing further demonstrate the potential of language models for analyzing legal and regulatory documents [29, 30].

In the regulatory compliance community, AI-assisted approaches have been explored for extracting obligations, identifying relevant requirements, checking completeness, supporting traceability, and detecting compliance violations [12, 13, 3, 5]. More recently, LLM-based approaches have been investigated for regulatory change detection and classification, showing that language models can support the identification of explicit and implicit changes in regulatory documents [16, 26]. Beyond regulatory change detection, recent LLM-based work in Requirements Engineering has also investigated automated change impact analysis. Etezadi et al. introduce ProReFiCIA, which combines LLM prompting with refinement and filtering stages to identify requirements affected by a change rationale, and further shows that incorporating domain knowledge through retrieval-augmented generation can improve impact identification [17]. While this demonstrates the potential of LLMs for semantic change propagation analysis, its focus is inter-requirement impact analysis rather than the analysis of regulatory changes and their downstream consequences across heterogeneous software artifacts.

Despite these promising developments, several limitations remain. AI models often require domain-specific adaptation, careful prompt design, and structured outputs to perform reliably in regulatory settings [29, 30, 26]. LLMs may also produce inconsistent outputs or hallucinated explanations, which makes human validation essential in high-stakes regulatory contexts [16, 26]. Prior work on automation further emphasizes that automation should be understood as support across different stages, such as information acquisition, information analysis, decision selection, and action implementation, rather than as full replacement of human decision-making [31, 26].

Most existing AI-supported approaches address individual tasks, such as extraction, classification, compliance checking, or change detection. However, regulatory engineering requires support for a broader workflow: identifying regulatory changes, interpreting their significance, linking them to affected software artifacts, and supporting impact assessment and decision-making. Consequently, there remains a need for systematic, human-in-the-loop approaches that combine regulatory change detection, change classification, and impact analysis while remaining grounded in industrial practice [5, 26, **?**].

1.2.4 Summary

The literature demonstrates considerable progress in Regulatory Requirements Engineering and AI-assisted support for regulatory compliance. Existing research has produced methods for compliance checking, requirements extraction, traceability recovery, legal document analysis, and more recently AI-assisted regulatory engineering [1, 21, 6, 3, 5, 16].

At the same time, the continuous evolution of regulations increases the need for approaches that support engineers beyond isolated compliance tasks. Existing work remains fragmented, often focusing on specific activities, regulations, domains, or artifact types. In particular, systematic support for regulatory change analysis and impact assessment remains comparatively underexplored, and empirical evaluations in realistic industrial settings are still limited [5, 7, 26, **?**]. These observations motivate the research gaps presented in the following section and form the foundation for the research presented in this dissertation.

1.3 Research Gaps

The background presented in the previous section shows that regulatory compliance has received increasing attention in software engineering and requirements engineering research. Existing work has

contributed methods for compliance checking, requirements extraction, traceability recovery, legal document analysis, and, more recently, AI-assisted regulatory analysis. However, the continuous evolution of regulations introduces challenges that are not yet sufficiently addressed. In particular, engineers need support not only for understanding regulations at a single point in time, but also for identifying regulatory changes, interpreting their relevance, and assessing their impact on software systems. Based on the literature and the empirical motivation of this dissertation, the following research gaps are addressed.

G1: Limited empirical understanding of regulatory engineering in highly regulated domains

Although Regulatory Requirements Engineering has received increasing research attention, empirical evidence on how regulatory requirements are handled in industrial practice remains limited [5]. Existing research reports a broad range of regulatory compliance challenges, but comparatively few studies investigate these challenges systematically through empirical research in industrial settings [5]. This limits our understanding of the practices and challenges experienced by practitioners, particularly in highly regulated domains where regulatory requirements constitute a recurring and evolving input to software development [7]. An in-depth understanding of such industrial contexts is important for grounding the development of methods and tools in actual engineering needs [7].

G2: Lack of structured conceptualization of regulatory changes

Regulatory changes are often treated as textual differences between document versions rather than as structured engineering-relevant phenomena. However, not all regulatory changes have the same meaning or impact. Some changes may be purely textual, while others introduce new data fields, procedural obligations, compliance checks, or policy-level requirements. Without a structured conceptualization of regulatory changes, it becomes difficult for engineers to reason consistently about their relevance and consequences. This also limits the development of automated techniques, since automation requires clear categories, definitions, and examples that can guide detection, classification, and interpretation [5, 26].

G3: Limited LLM-assisted support for regulatory change analysis

Traditional NLP techniques have provided important support for analyzing regulatory documents, for example through information extraction, classification, and compliance checking. However, such approaches are often designed for specific tasks and rely on predefined representations, rules, or task-specific training data. Recent advances in Large Language Models (LLMs), including general-purpose models and models specifically adapted to legal corpora, create new opportunities for analyzing regulatory changes at a semantic and contextual level. Their ability to process natural-language context and capture semantic relationships offers possibilities for moving beyond surface-level document comparison towards identifying and interpreting regulatory changes.

However, existing LLM-based approaches often focus on isolated regulatory engineering tasks. Little work has investigated how LLMs can support the broader regulatory change analysis process, including detecting changes, classifying their type, explaining their meaning, and supporting practitioners in determining whether further engineering action is needed. Moreover, the effectiveness of such techniques in realistic regulatory contexts remains insufficiently understood [16, 26].

G4: Limited support for regulatory impact analysis on software artifacts

Detecting a regulatory change is only the first step. The focus of this dissertation is not runtime compliance monitoring, but the evolution and maintenance of existing software systems in response to regulatory change. In practice, engineers must also determine how a regulatory change affects software artifacts such as requirements, workflows, architecture, implementation, documentation, and tests. Recent work has demonstrated that LLM-based approaches can support inter-requirement change impact analysis by identifying requirements affected by natural-language change rationales [17]. However, such approaches typically assume that the originating software change is already known and focus primarily on propagation among requirements. Existing work provides limited support for the additional steps required in a regulatory context, where changes must first be identified and interpreted from external

regulatory documents and subsequently related to heterogeneous software artifacts and engineering activities [17, 16]. This leaves a gap between identifying what changed in a regulation and understanding what needs to change in the software system.

Taken together, these gaps show that current research provides important foundations for Regulatory Requirements Engineering, but still offers limited support for handling evolving regulations as an integrated engineering process. This dissertation addresses these gaps by combining empirical studies, conceptual modeling, LLM-assisted change analysis, and tool-supported impact assessment. The following section presents the corresponding research goal and research questions.

1.4 Research Goal and Questions

Building upon the research gaps identified in the previous section, the overall goal of this dissertation is to:

Investigate the challenges software engineers face when analyzing evolving regulations and their impact on software systems, and the extent to which these challenges can be addressed through automated approaches.

Within this broader automation perspective, the dissertation particularly investigates Large Language Models as a means of supporting regulatory change analysis.

To achieve this goal, the dissertation adopts Design Science Research as an overarching methodological approach, integrating systematic literature studies and empirical investigations for problem understanding with solution design and implementation, followed by experimental and industrial evaluation.

The research is guided by the following research questions:

RQ1. *What are contemporary practices and challenges when handling frequently changing regulatory requirements in practical contexts?*

This question aims to establish an empirical understanding of current industrial practices, identify recurring challenges, and investigate opportunities for improving regulatory engineering processes.

RQ2. *How can regulatory changes be systematically represented and analyzed to support automation?*

This question investigates how regulatory changes can be conceptualized, categorized, and operationalized to provide a foundation for systematic and automated analysis. It further assesses the extent to which the resulting structured representation captures meaningful distinctions between regulatory changes and provides a useful basis for their subsequent analysis.

RQ3. *To what extent can we automate regulatory change analysis?*

This question investigates the extent to which regulatory change analysis can be automated, focusing on the detection, classification, and interpretation of regulatory changes. It assesses the effectiveness of LLM-based approaches across different types of regulatory changes, identifies which aspects of the analysis can be reliably supported through automation, and examines the limitations that require continued human involvement.

RQ4. *To what extent can a tool-supported approach translate detected regulatory changes into actionable software impact assessments, and how do practitioners evaluate its support for impact analysis?*

This question investigates how detected and categorized regulatory changes can be linked to affected software artifacts and corresponding engineering actions. It further evaluates the extent to which the resulting tool-supported approach supports practitioners in identifying and assessing relevant software impacts in an industrial setting.

Figure 1 illustrates the overall research program of this dissertation. It summarizes how the identified research gaps motivate the research goals and how these goals are addressed through the scientific contributions presented in the included papers. Together, the figure highlights the logical progression of the dissertation from problem identification to solution development and evaluation.

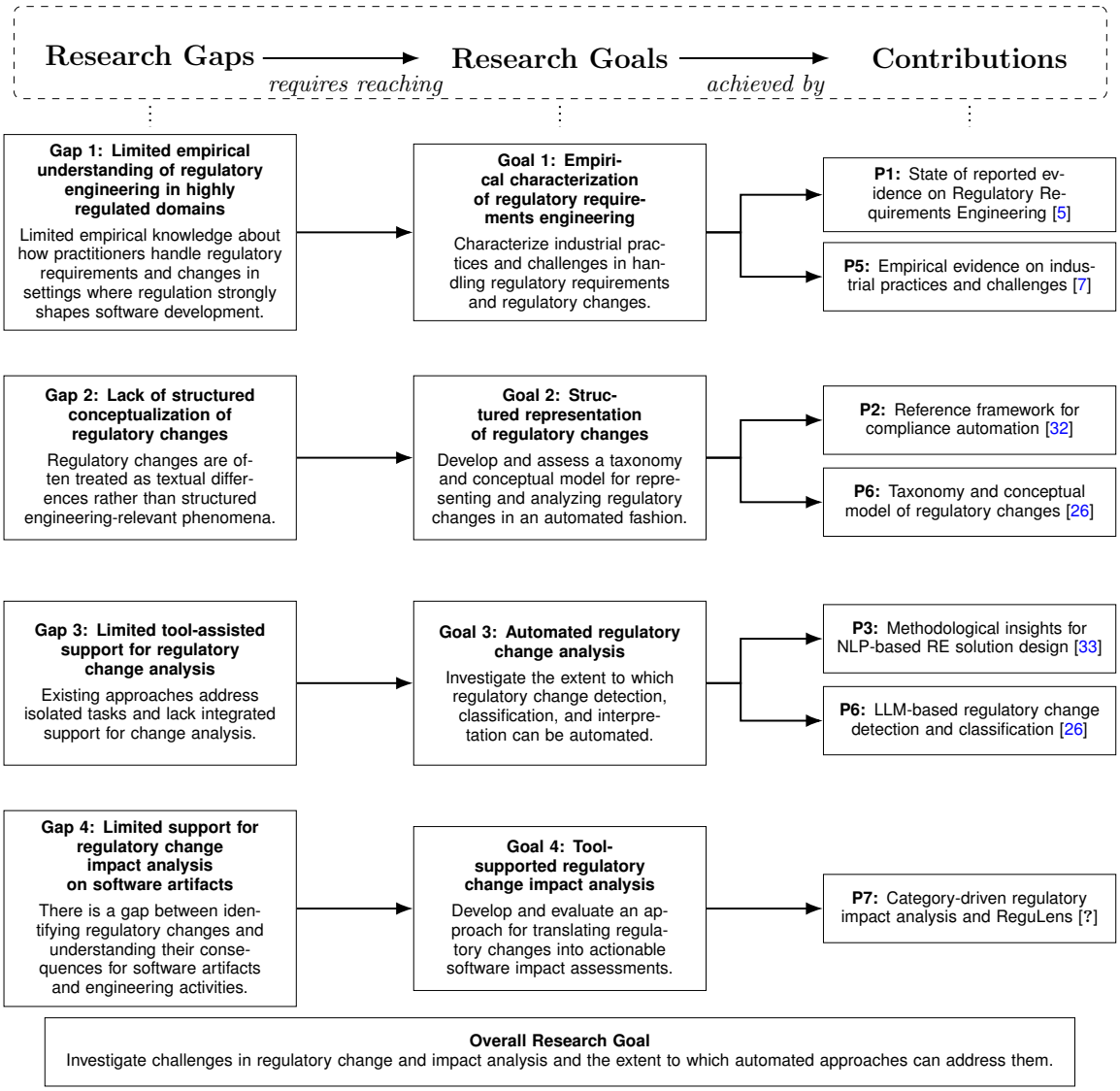


Figure 1: Research gaps, research goals, and corresponding contributions of this dissertation.

Table 1 further summarizes how the individual papers contribute to answering the overarching research questions. While each paper addresses its own study-specific objectives and research questions, the table shows how they collectively contribute to the overall research agenda of this dissertation.

1.5 Research Methodology

This dissertation is situated within the disciplinary area of empirical software engineering. As a guiding research approach, it draws on Design Science Research (DSR) as described by Wieringa [34]. DSR addresses practical problems through the iterative investigation of problems, the design and implementation of solutions, and the empirical evaluation of these solutions in their intended context. These activities are not necessarily performed as a linear sequence; rather, findings from problem investigation and evaluation can lead to refinements of both the understanding of the problem and the proposed solution.

Following this approach, the research in this dissertation combines three recurring activities. First, problem investigation is used to understand regulatory engineering practices, challenges, and research needs through literature studies and empirical investigations with practitioners. Second, solution design and implementation are used to develop conceptual and technical artifacts, including frameworks, taxonomies, LLM-assisted methods, and tool support. Third, the resulting artifacts are evaluated

Table 1: Mapping of the dissertation research questions to the included papers.

RQ	Research Question	Addressed by
RQ1	What are contemporary practices and challenges when handling frequently changing regulatory requirements in practical contexts?	P1, P4, P5
RQ2	How can regulatory changes be systematically represented and analyzed to support automation?	P2, P6
RQ3	To what extent can we automate regulatory change analysis?	P3, P6
RQ4	To what extent can a tool-supported approach translate detected regulatory changes into actionable software impact assessments, and how do practitioners evaluate its support for impact analysis?	P7

through empirical studies, experiments, and industrial case study to assess their ability to address the identified problems. The outcomes of these evaluations, in turn, inform subsequent problem understanding and solution refinement, reflecting the iterative nature of the research process.

Throughout the dissertation, empirical investigations and artifact development are tightly integrated. Industrial observations motivate the research, empirical findings guide the design of solution artifacts, and evaluations provide feedback for iterative refinement. Consequently, the included papers can be viewed as complementary studies that, despite evolving over time, collectively contribute to the dissertation's overarching goal of supporting regulatory change analysis and impact assessment.

1.5.1 Research Approach

The overall research is situated within Empirical Software Engineering, which emphasizes the systematic investigation of software engineering phenomena through observations, experiments, and industrial studies [35]. Empirical methods are used throughout this dissertation to understand current regulatory engineering practices, identify existing challenges, and evaluate the applicability of the proposed approaches in industrial settings.

To address the identified challenges, this dissertation adopts Design Science Research (DSR) as the overarching research methodology. Design Science focuses on creating artifacts that solve relevant problems while simultaneously generating scientific knowledge [36]. In this dissertation, these artifacts include conceptual models, taxonomies, LLM-assisted methods, and the ReguLens prototype for regulatory change analysis. The overall research process follows the Design Science Research Process Model proposed by Peffers et al. [37], while remaining iterative to accommodate continuous feedback from empirical investigations and industrial collaboration.

1.5.2 Design Science Research Process

To operationalize the DSR approach, the research activities are structured according to the six-phase Design Science Research Process Model proposed by Peffers et al. [37]. The phases were applied iteratively, allowing insights gained during design, demonstration, and evaluation to inform further problem investigation and refinement of the developed solutions.

Figure 2 illustrates the overall Design Science Research process followed throughout this dissertation and maps the individual papers to the six phases of the Design Science Research Process Model. While the phases are presented sequentially, the research evolved iteratively, with findings from evaluation continuously informing the refinement of conceptual models, methods, and artifacts.

Overall, the research progresses from understanding industrial problems towards developing and evaluating LLM-assisted methods and tool-supported approaches for regulatory change analysis. This iterative progression enables continuous refinement of both the conceptual foundation and the developed solution artifacts throughout the dissertation.

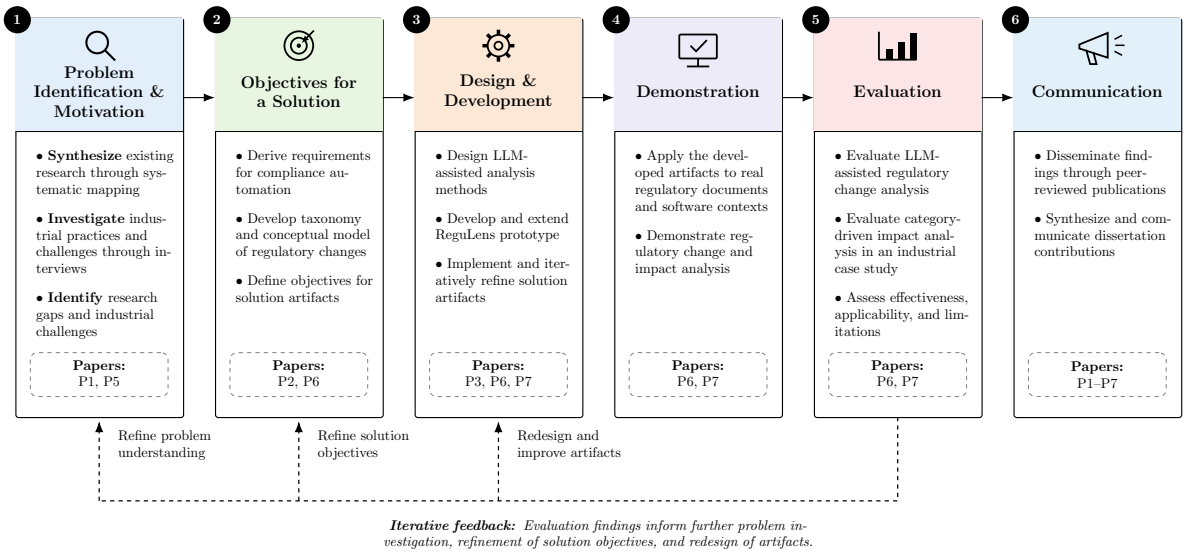


Figure 2: Design Science Research process followed in this dissertation and mapping of the included papers to the six DSR phases.

1.5.3 Research Methods

Different research methods were employed to address the research questions and provide evidence throughout problem investigation, solution development, and evaluation. These include secondary research through a systematic mapping study, qualitative empirical research through semi-structured interviews, experimental studies, and an industrial case study. The choice of method depended on the purpose of each study and the type of evidence required, with the individual papers describing the corresponding research procedures and methodological considerations in detail.

The research begins with a systematic mapping study following the guidelines proposed by Petersen et al. [38] to synthesize existing knowledge and identify research gaps within Regulatory Requirements Engineering. Industrial practices and challenges are subsequently investigated through semi-structured interviews with practitioners, following established qualitative research principles [39]. These empirical findings provide the problem understanding that motivates the development of solution artifacts.

Artifact development follows Design Science Research principles [36, 37]. Depending on the purpose and maturity of the respective artifact, evaluation is conducted through experimental studies and studies in industrial settings, with the resulting evidence informing subsequent refinement. The industrial case study follows the guidelines proposed by Runeson and Höst [40] to evaluate the applicability and usefulness of the proposed approach in a realistic software engineering context.

Table 2 summarizes the primary research methods and activities employed within the overarching DSR approach. At the dissertation level, DSR provides the overall structure for problem investigation, solution design and implementation, and evaluation. Individual papers may additionally report their work as DSR studies, reflecting the methodological framing adopted within those studies. In the synthesis presented here, however, the design and development activities of these studies are positioned as solution design and implementation within the overarching DSR process.

The combination of multiple research methods provides complementary forms of evidence across the dissertation. Qualitative and quantitative evidence, secondary research, solution design and implementation, and evaluation in experimental and industrial settings contribute to different aspects of the research questions. Where findings from different methods are brought together, their integration supports a more comprehensive understanding of the investigated problems and proposed solutions [41].

1.5.4 Research Context

The research presented in this dissertation was conducted in close collaboration with itestra GmbH, a software company that develops and maintains software systems for clients in highly regulated domains. The collaboration provided access to industrial expertise, regulatory engineering practices,

Table 2: Research methods and activities employed within the overarching DSR approach.

Method / Activity	Purpose	Papers
Systematic Mapping Study	Synthesize existing knowledge, identify research trends, and investigate research gaps.	P1
Semi-structured Interviews	Investigate industrial practices and practitioner challenges, and obtain practitioner feedback on developed conceptual and technical artifacts.	P5, P6, P7
Solution Design and Implementation	Design and develop candidate solutions, including conceptual models, taxonomies, LLM-assisted methods, and software tools.	P2, P3, P6, P7
Experimental Studies	Evaluate the performance and effectiveness of developed approaches under controlled conditions.	P6, P7
Industrial Case Study	Evaluate the applicability and usefulness of tool-supported regulatory impact analysis in an industrial setting.	P7

and realistic software engineering contexts in which regulatory requirements constitute an important and continuously evolving input to software development.

A substantial part of the empirical research focuses on the financial technology (FinTech) domain, particularly software systems operating in regulated financial and social security contexts. Within this setting, the German social security reporting regulation (DEÜV) and its associated implementation specifications constitute an important regulatory context for the studies on regulatory change analysis and impact assessment. These documents evolve over time and require corresponding analysis by practitioners to determine the relevance and potential consequences of regulatory changes for software systems.

The collaboration with itestra supported different stages of the research, including the investigation of industrial practices and challenges, the development and refinement of proposed solutions, and their evaluation in realistic industrial contexts. It therefore provided the practical setting in which the research problems and developed artifacts could be investigated throughout the DSR process.

1.5.5 Data Sources

The individual studies draw on multiple data sources corresponding to their respective research objectives. These sources include regulatory documents and change information, technical and software engineering artifacts, and empirical data collected from practitioners. Table 3 summarizes the main data sources and their role in the research.

Regulatory documents and associated change information provide the primary material for investigating regulatory evolution and evaluating approaches for regulatory change detection and classification. In particular, the German social security reporting regulation (DEÜV) and its associated implementation specifications provide concrete instances of evolving regulatory requirements and form an important empirical basis for the regulatory change analysis studies.

The research further draws on technical specifications and software engineering artifacts to investigate how identified regulatory changes relate to downstream engineering consequences. These sources provide the basis for developing and assessing the category-driven impact model and the tool-supported impact analysis approach.

Finally, practitioner interviews and workshops provide qualitative evidence about industrial practices, challenges, and practitioner perspectives on the proposed concepts and artifacts. Data collected during prototype and tool evaluations complement these sources by providing evidence about the use and perceived utility of the developed approaches in the investigated settings.

Table 3: Primary data sources used throughout the dissertation.

Data Source	Role in the Research
Regulatory documents and change information	Analysis of regulatory evolution and evaluation of regulatory change detection and classification
Implementation and technical specifications	Analysis of regulatory changes and their engineering implications
Practitioner interviews	Investigation of industrial practices and challenges and assessment of proposed concepts
Industrial workshops	Development, refinement, and assessment of proposed concepts and artifacts
Software engineering artifacts	Analysis of regulatory impacts and relationships between regulatory changes and software artifacts
Tool and prototype evaluation data	Assessment of the developed regulatory change and impact analysis approaches

1.5.6 Research Quality and Validity

Ensuring scientific rigor and practical relevance was a central objective throughout this dissertation. Within the overarching Design Science Research approach, multiple complementary research methods and forms of evidence were employed across problem investigation, solution design and implementation, and evaluation.

First, the combination of these methods provides complementary forms of evidence across the dissertation. The systematic mapping study contributes secondary evidence, semi-structured interviews and industrial studies provide qualitative empirical evidence, and experimental and industrial evaluations assess the developed conceptual and technical artifacts. Where findings from different methods are brought together, their integration supports a more comprehensive understanding of the investigated problems and proposed solutions [41].

Second, the research was conducted in close collaboration with industrial partners throughout the dissertation. Industrial practitioners contributed to problem identification, requirements elicitation, validation of the proposed taxonomy, refinement of the developed artifacts, and evaluation of the resulting approaches. This continuous collaboration helped ground the research in industrial problems and maintain its practical relevance.

Third, the iterative nature of the adopted Design Science Research approach supported the development and refinement of the dissertation's conceptual and technical contributions. Across the studies, findings from problem investigation, solution development, demonstration, and evaluation informed subsequent refinements to the understanding of the problem and to the developed artifacts [36, 37].

From a validity perspective, different validity considerations apply to the individual studies depending on their respective research designs. At the dissertation level, *construct validity* was supported by grounding the research questions, conceptual models, and evaluation criteria in existing literature and empirical evidence collected from industrial practice. *Internal validity* was considered through systematic research protocols, manually validated datasets, structured qualitative analysis, and verification procedures appropriate to the individual studies. *External validity* was addressed through empirical investigations and evaluations in realistic industrial settings. Although the empirical studies primarily focus on the German DEÜV ecosystem and the FinTech domain, the proposed engineering framework, taxonomy, LLM-assisted workflow, and impact analysis process were designed around concepts intended to be applicable beyond the investigated contexts. Their applicability to other regulated domains, however, requires further empirical evaluation. Finally, *reliability* was supported by documenting research procedures, datasets, annotation guidelines, prompts, experimental settings, and evaluation protocols in the included papers, enabling other researchers to understand, reproduce, and extend the presented work where industrial confidentiality permits.

Each included paper further discusses study-specific threats to validity and the corresponding mitigation strategies in relation to its particular research design. At the dissertation level, the combination

of empirical investigation, conceptual and technical artifact development, and evaluation provides complementary evidence for examining the research questions from different perspectives.

1.6 Research Contributions

This dissertation contributes empirical, conceptual, methodological, and technological knowledge to Regulatory Requirements Engineering in the context of evolving regulations. Although the individual studies addressed different research questions and evolved throughout the course of the dissertation, they collectively contribute to the overarching goal of investigating challenges in regulatory change and impact analysis and the extent to which automated and LLM-assisted approaches can address them. The research synthesizes existing knowledge and investigates industrial practice, establishes foundations for compliance automation, develops a structured representation of regulatory changes, investigates LLM-assisted regulatory change analysis, and extends this work towards tool-supported impact analysis on software artifacts.

The contributions are organized into five overarching areas. Each contribution synthesizes results from one or more included papers and is related to the research gaps and research questions introduced in Sections 1.3 and 1.4. Table 4 summarizes these relationships.

C1: Consolidated knowledge on Regulatory Requirements Engineering

The first contribution is a consolidated overview of the state of research on Requirements Engineering for regulatory compliance. The systematic mapping study synthesizes evidence from 280 primary studies and structures the literature according to reported compliance challenges, principles and practices, stakeholder involvement, software development process areas, regulatory fields, and application domains.

This contribution provides a broad knowledge base for understanding how Requirements Engineering contributes to regulatory compliance throughout the software development lifecycle. It also reveals that existing knowledge is fragmented across regulatory fields, domains, engineering activities, and stakeholder perspectives. The resulting classifications and mappings provide a foundation for positioning the subsequent empirical and design-oriented studies of this dissertation.

This contribution primarily addresses **G1** and supports **RQ1**. It is mainly provided by **P1**[5].

C2: Empirical understanding of regulatory engineering practices and challenges in the FinTech domain

The second contribution is an empirically grounded understanding of how regulatory requirements are handled in industrial software engineering practice. Through a multi-case investigation in the banking and insurance domain, the dissertation identifies current practices for interpreting regulations and inferring software requirements, as well as recurring challenges related to regulatory interpretation, documentation, traceability, communication, and system evolution.

The study further identifies activities in which practitioners perceive potential value from tool support, including regulatory document comparison, change notification, traceability recovery, and the identification of affected software components. At the same time, the findings emphasize that regulatory interpretation and consequential engineering decisions cannot be treated as fully automated activities and continue to require domain knowledge and human judgment.

This contribution grounds the subsequent solution development in industrial needs and reduces the risk of developing technically feasible but practically irrelevant approaches. It primarily addresses **G1** and **RQ1** and is mainly provided by **P5**[7].

C3: Engineering framework for human-centered compliance automation

The third contribution is an evidence-based reference framework for engineering compliance automation systems. The framework consolidates 57 domain-agnostic requirements synthesized from 207 requirement statements reported across 58 peer-reviewed studies. These requirements are organized into eight engineering categories:

- system goals,

- system quality,
- data quality,
- functionality,
- human interaction,
- integration,
- maintenance and evolution, and
- acceptance criteria.

The framework maps the identified requirements to four stages of automation: information acquisition, information analysis, decision and action selection, and action implementation. It further interprets the requirements through established human-centered automation principles concerning function allocation, human oversight, trust, workload, reliability, and the consequences of automation errors.

This contribution extends the dissertation beyond the functionality of individual AI techniques. It provides reusable guidance for designing, comparing, selecting, evaluating, and evolving compliance automation systems. It also shows that existing research emphasizes technical functionality and immediate operational goals, whereas acceptance, maintenance, human-centered risks, and long-term organizational consequences remain comparatively underexplored.

This contribution supports **RQ2** and **RQ3** and provides an engineering foundation for addressing **G2** and **G3**. It is primarily provided by **P2**[32].

C4: Structured and LLM-assisted regulatory change analysis

The fourth contribution is a structured and LLM-assisted approach to regulatory change analysis. It combines a conceptual representation of regulatory changes, empirical evidence from practitioners, automated change analysis, and their operationalization in the ReguLens[1] prototype.

First, the dissertation provides an empirically grounded taxonomy of regulatory changes derived from a manual analysis of Regulatory Implementation Specifications in the German social-security reporting domain. The taxonomy distinguishes five change scopes and four contextual dimensions, thereby providing a consistent vocabulary for describing recurring regulatory changes.

Second, the dissertation contributes practitioner evidence regarding the meaningfulness of the taxonomy and the challenges associated with handling different categories of regulatory change. The findings indicate that practitioners consider the taxonomy useful for filtering and organizing regulatory changes, while also identifying challenges related to ambiguity, traceability, and overlapping categories.

Third, the dissertation evaluates the feasibility of LLM-based support for detecting and classifying regulatory changes. The evaluation is based on a manually annotated gold-standard dataset containing 93 regulatory change instances and compares seven language models. The findings indicate that LLMs can support the identification and classification of recurring, structurally explicit changes, while performance decreases for narrative, weakly structured, or context-dependent changes. These limitations motivate a human-in-the-loop approach in which automated analysis supports rather than replaces expert judgment.

These conceptual and technical contributions are operationalized in ReguLens, a prototype that integrates regulatory change detection and categorization into a structured analysis workflow. Rather than relying on the language model alone, ReguLens combines LLM-based analysis with the regulatory change taxonomy and structured outputs to make detected changes available for subsequent engineering analysis. In this way, the prototype provides a bridge between the conceptual representation of regulatory changes and their automated analysis. Its extension towards category-driven and product-specific impact analysis is addressed in **C5**.

The design of the LLM-assisted solution is additionally informed by methodological lessons from the design science study reported in **P3**. That study provides insights into the iterative development of NLP-based RE solutions through problem investigation, prototyping, practitioner feedback, and refinement.

[1] https://regulens.site/

This contribution directly addresses **G2** and **G3** and answers **RQ2** and **RQ3**. It is primarily provided by **P6** [26], with methodological support from **P3** [33].

C5: Category-driven regulatory impact analysis

The fifth contribution extends regulatory change analysis from identifying *what changed* towards reasoning about *what the change may affect* in an existing software system. The focus is therefore on supporting software evolution and maintenance in response to regulatory change rather than runtime compliance monitoring. The dissertation develops a category-driven impact model that links regulatory change categories to potentially affected software artifacts, technical consequences, and typical engineering actions.

Based on this model, the dissertation extends ReguLens towards regulatory impact analysis by integrating three connected activities:

1. regulatory change detection and categorization,
2. category-driven impact assessment, and
3. product-specific impact analysis.

The underlying assumption is that recurring categories of regulatory change can provide structured information about potential impact patterns. ReguLens uses these patterns to derive implementation hypotheses and identify candidate software artifacts that can serve as starting points for further investigation. The approach is explicitly human-assisted: practitioners remain responsible for validating the identified impacts, interpreting product-specific conditions, and deciding on appropriate implementation actions.

The evaluation provides initial empirical evidence regarding the applicability and limitations of this approach in an industrial setting. The overall workflow from regulatory change identification and categorization to impact assessment and implementation investigation was considered understandable and aligned with the practitioner's reasoning process. ReguLens identified plausible entry points for implementation investigation, although it did not consistently identify all affected artifacts, and localization was more reliable at the repository or class level than at the method level.

The evaluation also revealed challenges that refine the underlying impact-analysis model. In particular, determining the appropriate granularity of regulatory changes emerged as an important issue. Complex regulatory amendments were better represented as coherent changes containing multiple implementation actions and applicability conditions than as collections of independent fine-grained changes. Furthermore, treating contextual conditions, such as effective dates or applicability constraints, as independent search units could introduce irrelevant implementation candidates. These findings suggest a need to distinguish more explicitly between implementation-relevant actions and contextual conditions when translating regulatory changes into product-specific impact hypotheses.

Overall, the evaluation positions ReguLens as a decision-support environment rather than an autonomous mechanism for determining the complete set of software changes required for compliance. Its contribution lies in supporting practitioners in structuring regulatory changes, reasoning about their potential engineering consequences, and identifying candidate locations for further investigation while retaining human expertise and judgment throughout the analysis process.

This contribution addresses **G4** and answers **RQ4**. It is primarily provided by **P7** [**?**].

Synthesis of Contributions

Taken together, the five contributions advance Regulatory Requirements Engineering through a progression from knowledge synthesis to tool-supported impact analysis. **C1** establishes the state of knowledge, and **C2** grounds the research problem in industrial practice. **C3** establishes reusable engineering requirements for developing human-centered compliance automation systems. **C4** provides the conceptual and technical foundations for structured, LLM-assisted regulatory change analysis through the regulatory change taxonomy and ReguLens. Finally, **C5** extends ReguLens towards category-driven and product-specific regulatory impact analysis.

The resulting contribution is not a fully autonomous compliance solution. Instead, the dissertation provides empirical evidence, conceptual structures, engineering guidance, LLM-assisted techniques,

Table 4: Mapping of the dissertation contributions to research gaps, research questions, and included papers.

ID	Contribution	Gaps	RQs	Papers
C1	Consolidated knowledge on Regulatory Requirements Engineering	G1	RQ1	P1
C2	Empirical understanding of regulatory engineering practice	G1	RQ1	P5
C3	Reference framework for human-centered compliance automation	G2, G3	RQ2, RQ3	P2
C4	Structured and LLM-assisted regulatory change analysis	G2, G3	RQ2, RQ3	P3, P6
C5	Category-driven regulatory impact analysis	G4	RQ4	P7

and tool support that enable practitioners to handle evolving regulations more systematically while retaining human responsibility for interpretation, impact validation, and implementation decisions.

1.7 Discussion

This dissertation addresses the challenge of supporting regulatory requirements engineering in environments where regulations evolve continuously and software systems must rapidly adapt to remain compliant. Rather than considering regulatory change analysis as an isolated activity, the dissertation views it as a continuous engineering process that combines empirical understanding, conceptual modeling, LLM-assisted methods, and software impact analysis.

The discussion reflects on the overall contributions of the dissertation, relates the findings to the identified research gaps, and outlines their implications for Regulatory Requirements Engineering research and industrial practice.

1.7.1 From Fragmented Activities to a Systematic Engineering Process

One of the central observations emerging from this dissertation is that regulatory change analysis has traditionally been treated as a collection of independent activities rather than as an integrated engineering process. Existing research predominantly focuses on individual tasks, such as extracting requirements from legal texts, identifying traceability links, or detecting textual differences. While these approaches address important problems, they provide limited support for understanding how regulatory changes propagate throughout software development.

The research presented in this dissertation demonstrates that regulatory change analysis can instead be viewed as a continuous engineering workflow consisting of several interconnected activities. The included studies collectively establish this workflow by progressively investigating industrial challenges, defining conceptual foundations, developing LLM-assisted methods, and linking detected changes to software artifacts.

Consequently, the dissertation shifts the perspective from isolated automation techniques towards an integrated view of Regulatory Requirements Engineering that spans the complete lifecycle of regulatory evolution.

This progression also reflects the cumulative and iterative research trajectory of the dissertation. The individual studies were not conceived as a strictly sequential research program, but findings and experiences from earlier studies informed subsequent research directions. The systematic mapping study (P1) synthesized existing knowledge and identified research gaps, while the industrial study (P5) provided empirical evidence about practitioners' challenges and automation needs. P2 contributed engineering requirements for compliance automation, while P6 developed a taxonomy and conceptual model of regulatory changes and investigated LLM-assisted change analysis. Methodological insights from P3 further informed the development of NLP- and LLM-assisted RE solutions. Finally, P7 extended the regulatory change analysis work towards category-driven software impact analysis. Taken together, these studies form an iterative trajectory from problem understanding through solution development towards empirical evaluation and refinement.

1.7.2 The Importance of Conceptual Foundations

A second contribution concerns the role of conceptual modeling within Regulatory Requirements Engineering. Before regulatory changes can be automated, they must first be understood.

The empirical investigations revealed that practitioners reason about regulatory changes using implicit domain knowledge that is rarely documented explicitly. Existing literature similarly lacks a common conceptual vocabulary for describing different categories of regulatory changes. This absence of shared concepts limits communication between stakeholders while simultaneously hindering the development of automated approaches.

The taxonomy and conceptual model developed in this dissertation therefore constitute more than classification schemes. They provide an abstraction layer between natural-language regulations and LLM-assisted analysis methods. Similar to taxonomies developed in other areas of Requirements Engineering, they establish a common language that enables systematic reasoning, comparison across studies, and reproducible automation.

This conceptual foundation also illustrates that effective LLM-assisted approaches require domain knowledge rather than relying solely on increasingly capable language models.

1.7.3 Human-Centered LLM Support for Regulatory Requirements Engineering

Another important outcome of the dissertation concerns the role of LLMs as decision-support mechanisms rather than replacements for engineering expertise. The findings indicate that LLMs can support repetitive and time-consuming regulatory engineering activities, particularly regulatory change detection, classification, and preliminary analysis. At the same time, regulatory interpretation and impact assessment remain knowledge-intensive activities that depend on domain knowledge, organizational context, and engineering judgment.

Large Language Models and Natural Language Processing techniques show considerable potential for automating repetitive and time-consuming regulatory engineering activities. The developed ReguLens prototype provides evidence that LLM-assisted methods can support regulatory change detection, classification, and preliminary impact analysis. The industrial evaluation further indicates that such support can provide practitioners with structured starting points for implementation investigation, while also showing that domain knowledge and engineering judgment remain necessary for validating and refining the identified impacts.

However, the empirical findings equally indicate that regulatory engineering remains a knowledge-intensive activity requiring human expertise. Regulatory interpretation frequently depends on organizational context, legal understanding, and engineering judgment that cannot be fully captured through automated approaches alone.

Accordingly, the dissertation advocates a human-centered perspective on LLM-assisted Regulatory Requirements Engineering in which LLM-assisted methods support engineers by reducing manual effort while final compliance decisions remain under human responsibility. This perspective aligns with current developments in trustworthy and human-centered artificial intelligence and provides a realistic pathway for industrial adoption.

1.7.4 Bridging Regulatory Changes and Software Engineering

The dissertation further extends existing research by explicitly connecting regulatory evolution with software engineering artifacts.

Existing research has investigated how changes propagate across software requirements. For example, ProReFiCIA demonstrates that LLMs can identify requirements affected by natural-language change rationales while substantially reducing the number of requirements that analysts must inspect [17]. Such work provides important evidence that LLMs can support semantic change propagation in Requirements Engineering. In practice, however, software organizations are primarily interested in understanding the consequences of these changes not only for requirements but also for architecture, implementation, testing, and maintenance activities. The proposed category-driven impact analysis approach addresses this limitation by linking detected regulatory changes to software engineering artifacts and development activities. Consequently, regulatory changes become actionable engineering information rather than isolated textual modifications.

This integration contributes toward treating regulatory evolution as a first-class concern within Requirements Engineering and software evolution research.

1.7.5 Beyond General-Purpose Generative AI

Unlike general-purpose LLMs that answer prompts independently, ReguLens embeds LLMs within a structured regulatory engineering workflow consisting of domain-specific taxonomies, reproducible prompting strategies, category-driven impact reasoning, traceability support, and human validation.

More importantly, the scientific contribution of ReguLens does not reside in the underlying language model itself. Foundation models will continue to evolve, and future models may outperform those evaluated in this dissertation. The lasting contribution instead lies in the engineering framework surrounding the model. ReguLens combines domain-specific knowledge, a structured taxonomy of regulatory changes, reproducible prompting strategies, standardized outputs, category-driven impact reasoning, traceability support, and explicit human validation within a coherent regulatory engineering workflow. These components operationalize general-purpose LLM capabilities into a repeatable, explainable, and auditable engineering assistant that supports practitioners throughout the regulatory change analysis process rather than merely generating answers to individual prompts.

Consequently, ReguLens should not be viewed as an alternative to foundation models such as GPT or Claude, but as a framework that operationalizes such models for Regulatory Requirements Engineering. As language models continue to improve, the underlying AI component can be replaced without changing the conceptual workflow, engineering process, or human-centered principles developed in this dissertation.

In this sense, the scientific contribution of ReguLens is largely model-agnostic: its value lies in organizing regulatory engineering knowledge and integrating AI into a structured software engineering process rather than depending on the capabilities of any particular foundation model.

1.7.6 Implications

The findings and discussion above have implications for both future research and industrial practice. The following sections summarize these implications from the perspectives of researchers and practitioners.

Implications for Research The findings of this dissertation suggest several implications for future research in Regulatory Requirements Engineering.

First, empirical investigations remain essential for understanding industrial regulatory engineering practices. Many existing assumptions regarding regulatory compliance are insufficiently validated in industrial settings. Additional empirical studies across different regulated domains would therefore strengthen the external validity of current knowledge.

Second, future LLM-assisted approaches should increasingly combine domain-specific conceptual knowledge with modern language models. Rather than viewing Large Language Models as standalone solutions, future research should investigate hybrid approaches that integrate ontologies, taxonomies, traceability information, and domain knowledge.

Third, research should increasingly consider regulatory change analysis as part of a broader software engineering ecosystem. Future work should investigate how automated regulatory analysis integrates with requirements management, architecture evolution, testing, DevOps pipelines, and software maintenance processes.

Finally, benchmarking datasets, shared evaluation protocols, and publicly available industrial case studies would improve reproducibility and facilitate comparison between emerging LLM-assisted approaches.

Implications for Industrial Practice For industrial practice, the findings indicate opportunities for using LLM-assisted regulatory change analysis as decision support within existing engineering processes. Practitioners currently invest considerable effort in identifying, interpreting, and assessing regulatory changes, and the approaches investigated in this dissertation show how parts of this process can be supported while retaining human responsibility for interpretation and implementation decisions.

The proposed taxonomy provides a structured way to organize regulatory changes, while the LLM-assisted methods support their identification and classification. ReguLens further connects these activities with category-driven impact assessment and product-specific investigation, providing practitioners with structured starting points for analyzing the potential consequences of regulatory changes for software artifacts.

The findings also emphasize that such support should augment rather than replace practitioner expertise. Regulatory interpretation and software impact assessment remain dependent on domain knowledge, product context, and engineering judgment. Consequently, the practical role of ReguLens is best understood as supporting practitioners in filtering and structuring regulatory changes, developing impact hypotheses, and identifying candidate areas for further investigation.

1.7.7 Limitations

While this dissertation advances the state of the art in LLM-assisted RRE, several limitations should be acknowledged.

Domain scope. The empirical investigations and evaluations were conducted primarily within the German social security reporting ecosystem (DEÜV) and in collaboration with industrial partners from the FinTech domain. This environment provides a realistic and representative case of continuously evolving regulations, but it also limits the direct generalizability of the findings to other regulatory ecosystems. Although the proposed framework, taxonomy, LLM-assisted methods, and impact analysis approach were designed to capture domain-independent concepts, their applicability to other highly regulated domains remains to be empirically validated.

Data availability. The research relies on industrial regulatory documents, technical specifications, practitioner interviews, and software engineering artifacts. As is common in regulatory engineering research, much of the underlying industrial data is confidential and cannot be publicly released. Consequently, the reproducibility of some empirical studies is constrained by limited access to industrial datasets. Furthermore, the evaluation corpus focuses on a specific regulatory ecosystem and language, which may not fully capture the diversity of regulatory documents encountered in other jurisdictions, languages, or application domains.

Evaluation scope. The proposed methods and the ReguLens prototype have been evaluated through empirical studies, experimental investigations, and an industrial case study. These evaluations provide evidence regarding the feasibility, applicability, and limitations of the proposed approaches, but they do not constitute long-term deployment studies. The current industrial evaluation therefore provides initial evidence about how ReguLens can support regulatory impact analysis in a realistic software engineering setting rather than evidence of sustained organizational adoption or long-term impact on software engineering practice. Further longitudinal evaluations are needed to assess how LLM-assisted regulatory engineering influences productivity, decision quality, collaboration, and organizational processes over extended periods.

These limitations do not diminish the contributions of the dissertation but rather define the current boundaries of the research and motivate the directions for future work presented below.

1.7.8 Future Work

Several directions remain for extending the research presented in this dissertation.

First, the empirical studies and evaluations were primarily conducted in the FinTech domain and within the German social-security reporting ecosystem. This setting provides a realistic example of a complex and continuously evolving regulatory environment, but it also limits the extent to which the findings can be generalized directly to other contexts. Future studies should investigate the applicability of the proposed framework, taxonomy, LLM-assisted methods, and impact model in other highly regulated domains, including healthcare, medical devices, automotive systems, railway systems, aviation, energy, insurance, and public administration. Cross-domain studies could distinguish regulatory change concepts that are broadly applicable from those requiring domain-specific adaptation.

Second, future work should extend and validate the regulatory change taxonomy using additional regulations, document structures, languages, and jurisdictions. The current taxonomy provides a foundation for systematic analysis, but its categories and contextual dimensions may evolve as new regulatory ecosystems are studied. Longitudinal validation would also help determine whether the taxonomy remains stable as regulations, organizational practices, and software technologies change.

Third, further research is needed to evaluate the long-term industrial effectiveness of ReguLens. In particular, future studies should investigate its influence on analysis effort, precision and recall of

identified impacts, consistency between practitioners, time required to implement regulatory changes, user trust, and the quality of final engineering decisions. Longitudinal studies embedded in ongoing regulatory projects would provide stronger evidence than isolated evaluations and would reveal how AI-assisted support affects organizational practices over time.

Fourth, regulatory change analysis should be integrated more closely with existing software engineering environments. Future versions of ReguLens could connect with requirements management systems, issue trackers, architecture repositories, test-management platforms, source-code repositories, and DevOps pipelines. Such integration could enable continuous regulatory monitoring, automated creation of candidate traceability links, impact alerts, and coordinated updates across software artifacts.

Fifth, future work should investigate hybrid LLM-based architectures that combine Large Language Models with retrieval-augmented generation, knowledge graphs, rule-based validation, regulatory ontologies, historical change repositories, and product-specific traceability information. These hybrid approaches may improve consistency, explainability, and grounding while reducing hallucinations and unsupported interpretations. They may also allow ReguLens to distinguish between general regulatory impact patterns and consequences that are specific to a particular product or organization.

Sixth, further research is needed on human-AI collaboration in regulatory engineering. Relevant questions include how explanations should be presented, how uncertainty should be communicated, which decisions should remain exclusively human, and how practitioners can efficiently inspect and correct automated results. The reference framework developed in this dissertation provides a foundation for such studies, particularly regarding trust, workload, human oversight, acceptance, and responsibility allocation.

Finally, future work should establish shared datasets, benchmarks, and evaluation protocols for regulatory change analysis and impact assessment. Publicly available annotated datasets remain limited, and industrial regulatory documents and software artifacts are often confidential. Shared benchmarks would make it easier to compare alternative approaches, reproduce experimental results, and assess progress independently of the underlying language model.

1.7.9 Discussion Summary

Taken together, the contributions presented in this dissertation show how empirical software engineering, conceptual modeling, and Design Science Research can be combined to investigate regulatory change analysis and develop corresponding engineering support. The research follows an iterative trajectory from understanding industrial challenges towards developing and evaluating structured, LLM-assisted, and tool-supported approaches.

The overall contribution extends beyond individual methods or tools. It provides a systematic foundation for treating regulatory change analysis as a continuous software engineering activity supported by empirical evidence, conceptual models, LLM-assisted methods, and human expertise.

Rather than viewing Large Language Models as replacements for domain expertise, the findings indicate that their value lies in being embedded within structured engineering processes. The contribution of ReguLens therefore lies not in the underlying model itself, but in operationalizing general-purpose LLM capabilities within a reproducible, domain-informed, and human-assisted workflow for regulatory change and impact analysis.

1.8 Conclusion

Software-intensive systems operating in regulated environments must continuously adapt to evolving regulatory requirements. However, regulatory change handling remains a predominantly manual, knowledge-intensive, and fragmented engineering activity. Engineers must identify relevant changes, interpret their meaning, determine their significance, and assess their consequences for software requirements, workflows, architecture, implementation, documentation, and testing. This dissertation investigated how these activities can be supported systematically through empirical evidence, conceptual structures, automated analysis, and LLM-assisted methods.

The overall goal of this dissertation was to investigate the challenges software engineers face when analyzing evolving regulations and their impact on software systems, and the extent to which these challenges can be addressed through automated and LLM-assisted approaches. To address this goal,

the dissertation followed an iterative Design Science Research approach combining systematic literature synthesis, industrial investigations, conceptual modeling, solution design and implementation, experimental studies, and evaluation in an industrial setting.

The research first consolidated the existing body of knowledge on Regulatory Requirements Engineering and identified the fragmentation of current research across regulations, application domains, engineering activities, and stakeholder perspectives. It then established an empirical understanding of how regulatory requirements and regulatory changes are handled in industrial practice, including the recurring challenges associated with interpretation, documentation, traceability, coordination, and software evolution.

Building on this empirical foundation, the dissertation developed an evidence-based engineering framework for human-centered compliance automation. The framework provides reusable requirements for designing, comparing, evaluating, and evolving compliance automation systems. It emphasizes that effective automation must address not only technical functionality, but also system quality, data quality, human interaction, integration, maintainability, acceptance, and the allocation of responsibility between humans and automated systems.

The dissertation further developed a taxonomy and conceptual model of regulatory changes that provide a consistent vocabulary for describing recurring change phenomena. This conceptual structure supports both human reasoning and automated analysis by distinguishing different forms of regulatory change and the contextual dimensions required for interpreting them. The taxonomy was subsequently operationalized through LLM-based methods for detecting and classifying regulatory changes.

The findings show that LLMs can provide meaningful support for identifying and classifying recurring and structurally explicit regulatory changes. At the same time, performance decreases when changes are narrative, weakly structured, ambiguous, or dependent on broader regulatory and organizational context. These findings reinforce the central position of this dissertation: LLM-assisted regulatory engineering should augment human expertise rather than replace it. Practitioners remain responsible for validating interpretations, resolving ambiguity, assessing organizational relevance, and making implementation decisions.

Finally, the dissertation extends regulatory change analysis from identifying *what changed* toward reasoning about *what the change may affect*. The category-driven impact model and the ReguLens prototype connect detected regulatory changes with potentially affected software artifacts, technical consequences, and engineering activities. The industrial evaluation provides initial evidence that this workflow can support practitioners by providing structured starting points for implementation investigation, while also revealing important limitations. In particular, the evaluation showed that the granularity at which regulatory changes are represented affects subsequent impact analysis, that contextual conditions and implementation actions need to be distinguished more explicitly, and that candidate localization is more reliable at broader artifact levels than at fine-grained method level. These findings position ReguLens as a decision-support environment that assists practitioners in structuring regulatory changes, developing impact hypotheses, and identifying candidate areas for further investigation rather than autonomously determining the complete software impact of a regulatory change.

Taken together, the dissertation advances Regulatory Requirements Engineering in five complementary ways. It contributes:

1. a consolidated understanding of the existing Regulatory Requirements Engineering body of knowledge;
2. empirical evidence concerning regulatory engineering practices, challenges, and automation opportunities;
3. an engineering framework for human-centered compliance automation systems;
4. a structured and LLM-assisted approach to regulatory change analysis; and
5. a category-driven and tool-supported approach to regulatory impact analysis on software artifacts.

The overall contribution is therefore not a fully autonomous compliance solution. Rather, the dissertation provides empirical knowledge, conceptual foundations, engineering guidance, LLM-assisted methods, and tool support for making regulatory change analysis more systematic, transparent, and

repeatable. Unlike approaches that address individual compliance tasks, the dissertation investigates an integrated engineering process spanning regulatory change identification, conceptualization, LLM-assisted analysis, and software impact assessment. The scientific contribution of ReguLens does not depend on any single foundation model. Instead, it lies in embedding general-purpose LLM capabilities within a domain-informed and reproducible engineering workflow that incorporates regulatory change categories, structured outputs, traceability, impact reasoning, and human validation.

1.8.1 Closing Perspective

Regulatory compliance will continue to evolve as regulations, technologies, and software-dependent services become increasingly interconnected. The challenge for software engineering is therefore not merely to interpret individual regulations, but to establish sustainable processes for continuously identifying, understanding, and implementing regulatory change.

This dissertation takes a step toward that objective by demonstrating how empirical software engineering, conceptual modeling, Design Science Research, and LLM-assisted methods can be combined to support regulatory change analysis and impact assessment. Its central conclusion is that the most promising role of LLMs in Regulatory Requirements Engineering is not to replace legal, domain, or software engineering expertise, but to support practitioners in organizing information, automating repetitive analysis, identifying potential impacts, and making more informed engineering decisions.

By embedding LLM capabilities within a structured and human-centered engineering process, the dissertation contributes toward a future in which regulatory evolution is addressed not as a reactive and isolated maintenance task, but through a systematic and tool-supported process for identifying regulatory changes and assessing their consequences for existing software systems and engineering artifacts.

References

[1] T. Breaux, A. Antón, Analyzing regulatory rules for privacy and security requirements, IEEE transactions on software engineering 34 (1) (2008) 5–20.

[2] P. N. Otto, A. I. Antón, Addressing legal requirements in requirements engineering, in: 15th IEEE international requirements engineering conference (RE 2007), IEEE, 2007, pp. 5–14.

[3] J. P. Castellanos Ardila, B. Gallina, F. Ul Muram, Compliance checking of software processes: A systematic literature review, Journal of Software: Evolution and Process 34 (5) (2022) e2440.

[4] M. Mubarkoot, J. Altmann, M. Rasti-Barzoki, B. Egger, H. Lee, Software compliance requirements, factors, and policies: A systematic literature review, Computers & Security 124 (2023) 102985.

[5] O. Kosenkov, P. Elahidoost, T. Gorschek, J. Fischbach, D. Mendez, M. Unterkalmsteiner, D. Fucci, R. Mohanani, Systematic mapping study on requirements engineering for regulatory compliance of software systems, Information and Software Technology (2025). doi:https://doi.org/10.1016/j.infsof.2024.107622.

[6] S. Ghanavati, A. Rifaut, E. Dubois, D. Amyot, Goal-oriented compliance with multiple regulations, in: 2014 IEEE 22nd international requirements engineering conference (RE), IEEE, 2014, pp. 73–82.

[7] P. Elahidoost, D. Mendez, M. Unterkalmsteiner, J. Fischbach, C. Feiler, J. Streit, Practices, challenges, and opportunities when inferring requirements from regulations in the fintech sector - an industrial study, in: 2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW), 2024.

[8] J. Bosch, Continuous software engineering, in: Proceedings of the International Conference on Software Engineering Companion, 2014.

[9] B. Fitzgerald, K.-J. Stol, Continuous software engineering: A roadmap and agenda, Journal of Systems and Software 123 (2017) 176–189.

[10] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, H. Wang, Large language models for software engineering: A systematic literature review, ACM Transactions on Software Engineering and Methodology 33 (8) (2024). doi:10.1145/3695988.

[11] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, J. M. Zhang, Large language models for software engineering: Survey and open problems, arXiv preprint arXiv:2310.03533 (2023).

[12] N. Zeni, N. Kiyavitskaya, L. Mich, J. Cordy, J. Mylopoulos, Gaiust: supporting the extraction of rights and obligations for regulatory compliance, Requirements Engineering 20 (2015) 1–22.

[13] A. Sleimi, N. Sannier, M. Sabetzadeh, L. Briand, M. Ceci, J. Dann, An automated framework for the extraction of semantic legal metadata from legal texts, Empirical Software Engineering (2021).

[14] O. A. Cejas, S. Abualhaija, L. C. Briand, Compai: A tool for gdpr completeness checking of privacy policies using artificial intelligence, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, 2024.

[15] S. Abualhaija, C. Arora, L. C. Briand, Coreqqa: a compliance requirements understanding using question answering tool, in: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2022.

[16] S. Abualhaija, M. Ceci, N. Sannier, D. Bianculli, L. C. Briand, D. Zetzsche, M. Bodellini, Ai-enabled regulatory change analysis of legal requirements, in: 2024 IEEE 32nd International Requirements Engineering Conference (RE), 2024.

[17] R. Etezadi, S. Abualhaija, C. Arora, L. Briand, Llm-driven cost-effective requirements change impact analysis (2026). arXiv:2511.00262.
URL https://arxiv.org/abs/2511.00262

[18] N. S. Abdullah, S. Sadiq, M. Indulska, Emerging challenges in information systems research for regulatory compliance management, in: Proceedings of the 22nd International Conference on Advanced Information Systems Engineering, CAiSE'10, Springer-Verlag, Berlin, Heidelberg, 2010, p. 251–265.

[19] A. Cleven, R. Winter, Regulatory compliance in information systems research literature analysis and research agenda, http://www.alexandria.unisg.ch/Publikationen/67909 29 (01 2009).

[20] O. Kosenkov, Towards viewpoint-centric artifact-based regulatory requirements engineering for compliance by design (2026). arXiv:2603.09492.
URL https://arxiv.org/abs/2603.09492

[21] A. Siena, A. Perini, A. Susi, J. Mylopoulos, Towards a framework for law-compliant software requirements, in: 2009 31st International Conference on Software Engineering-Companion Volume, IEEE, 2009, pp. 251–254.

[22] Publications Office of the European Union, Legal acts – statistics (2026).
URL https://eur-lex.europa.eu/statistics/2026/legislative-acts-statistics.html

[23] O. Al-Ubaydli, P. A. McLaughlin, Regdata: A numerical database on industry-specific regulations for all united states industries and federal regulations, 1997–2012, Regulation & Governance 11 (1) (2017) 109–123. doi:https://doi.org/10.1111/rego.12107.
URL https://onlinelibrary.wiley.com/doi/abs/10.1111/rego.12107

[24] S. Jayatilleke, R. Lai, K. Reed, Managing software requirements changes through change specification and classification, Computer Science and Information Systems (2017).

[25] C. Arora, M. Sabetzadeh, A. Goknil, L. Briand, F. Zimmer, Change impact analysis for natural language requirements: an nlp approach, in: 2015 IEEE 23rd International Requirements Engineering Conference, RE 2015 - Proceedings, IEEE, Institute of Electrical and Electronics Engineers, United States of America, 2015, pp. 6–15, publisher Copyright: © 2015 IEEE.; IEEE

International Requirements Engineering Conference 2015, RE 2015 ; Conference date: 24-08-2015 Through 28-08-2015. doi:10.1109/RE.2015.7320403.
URL https://ieeexplore.ieee.org/xpl/conhome/7310793/proceeding

[26] P. Elahidoost, H. Villamizar, F. Angermeir, J. Streit, D. Mendez, M. Unterkalmsteiner, T. Gorschek, Investigating automated change analysis in fintech regulations, Inf. Softw. Technol. 195 (C) (Jul. 2026). doi:10.1016/j.infsof.2026.108144.
URL https://doi.org/10.1016/j.infsof.2026.108144

[27] S. Maro, J.-P. Steghöfer, M. Staron, Software traceability in the automotive domain: Challenges and solutions, Journal of Systems and Software (2018).

[28] M. Barrientos, K. Winter, S. Rinderle-Ma, Impact analysis of regulatory requirement changes on business process compliance, Information and Software Technology 194 (2026) 108079. doi:https://doi.org/10.1016/j.infsof.2026.108079.
URL https://www.sciencedirect.com/science/article/pii/S0950584926000686

[29] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, I. Androutsopoulos, Legal-bert: The muppets straight out of law school, in: Findings of the Association for Computational Linguistics (ACL), 2021.

[30] T. Deußer, C. Zhao, L. Sparrenberg, D. Uedelhoven, A. Berger, M. Pielka, L. Hillebrand, C. Bauckhage, R. Sifa, A comparative study of large language models for named entity recognition in the legal domain, in: 2024 IEEE International Conference on Big Data (BigData), IEEE, 2024.

[31] R. Parasuraman, T. Sheridan, C. Wickens, A model for types and levels of human interaction with automation, IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans 30 (3) (2000) 286–297.

[32] F. Angermeir, P. Elahidoost, F. Moyon, T. Gorschek, D. M. Fernandez, A reference framework of requirements for the automation of compliance systems (2026). doi:10.13140/RG.2.2.31147.81443.

[33] P. Elahidoost, M. Unterkalmsteiner, D. Fucci, P. Liljenberg, J. Fischbach, Designing nlp-based solutions for requirements variability management: Experiences from a design science study at visma, in: D. Mendez, A. Moreira (Eds.), Requirements Engineering: Foundation for Software Quality, Springer Nature Switzerland, Cham, 2024, pp. 191–204.

[34] R. Wieringa, Design Science Methodology for Information Systems and Software Engineering, 2014. doi:10.1007/978-3-662-43839-8.

[35] C. Wohlin, P. Runeson, M. Höst, M. Ohlsson, B. Regnell, A. Wesslén, Experimentation in Software Engineering, Springer, 2012.

[36] A. R. Hevner, S. T. March, J. Park, S. Ram, Design science in information systems research, MIS Q. 28 (1) (2004) 75–105.

[37] K. Peffers, T. Tuunanen, M. Rothenberger, S. Chatterjee, A design science research methodology for information systems research, Journal of Management Information Systems 24 (2007) 45–77.

[38] K. Petersen, R. Feldt, S. Mujtaba, M. Mattsson, Systematic mapping studies in software engineering, in: Proceedings of the 12th International Conference on Evaluation and Assessment in Software Engineering, EASE'08, BCS Learning & Development Ltd., Swindon, UK, 2008, pp. 68–77.

[39] S. Kvale, S. Brinkmann, InterViews: Learning the Craft of Qualitative Research Interviewing, SAGE, 2009.

[40] P. Runeson, M. Höst, Guidelines for conducting and reporting case study research in software engineering, Empirical Software Engineering (2009).

[41] M.-A. Storey, R. Hoda, A. Maciel Paz Milani, M. T. Baldassarre, Guiding principles for mixed methods research in software engineering, Empirical Software Engineering 30 (5) (2025) 138. doi:10.1007/s10664-025-10629-x.